高斯模糊

高斯模糊
- 例程效果
- 原理讲解
  - 什么是图像高斯滤波
- 代码概述
  - 导入模块
    - 设置图像尺寸
    - 初始化摄像头（RGB888 格式）
    - 初始化显示模块
    - 初始化媒体管理器并启动摄像头
    - 设置高斯滤波参数
    - 图像处理与高斯滤波
    - 资源释放
  - 参数调整说明

例程效果

运行本节例程代码 [源码汇总/17.cv_lite/17.rgb888_gaussian_blur.py]

本节我们将结合 `cv_lite` 扩展模块在嵌入式设备上实现 RGB888 格式的图像高斯滤波处理。



原理讲解

什么是图像高斯滤波

图像高斯滤波（Gaussian Blur）是一种常用的图像平滑技术，用于减少图像中的噪声并模糊细节。它基于高斯分布（正态分布）设计滤波核，通过对图像进行卷积操作来实现平滑效果。高斯滤波在图像处理中广泛应用，特别是在预处理阶段。

- **操作原理**：高斯滤波使用一个基于高斯分布的卷积核（或称权重矩阵），对图像中的每个像素及其周围像素进行加权平均。核的中心具有最高的权重，权重随着距离中心增加而逐渐减小（符合正态分布）。这种加权平均使得像素值变化更平滑，细节被模糊，噪声被抑制。
- **效果**：高斯滤波能够有效去除图像中的高频噪声（如随机噪点），同时保留图像的低频信息（如整体结构和大的边缘）。与均值滤波相比，高斯滤波更自然，因为它更接近人眼的视觉感知，不会引入明显的方块状伪影。
- **应用场景**：高斯滤波常用于图像去噪、边缘检测前的预处理（减少噪声对边缘检测的干扰）、图像降采样（防止混叠）以及特效处理（如背景虚化）。

在高斯滤波中，卷积核的大小（必须为奇数）会显著影响结果：较大的核会导致更强的模糊效果，但计算量也会增加。此外，高斯滤波的参数还可能包括标准差（sigma），用于控制核的权重分布（本代码中未显式指定，可能是固定值或由库内部处理）。

代码概述

导入模块

```
import time, os, sys, gc
from machine import Pin
from media.sensor import *    # 摄像头接口 / Camera interface
from media.display import *   # 显示接口 / Display interface
from media.media import *     # 媒体资源管理器 / Media manager
import _thread
import cv_lite                # cv_lite扩展模块 / cv_lite extension (includes
gaussian_blur API)
import ulab.numpy as np       # ulab 数组模块 / NumPy-like ndarray for MicroPython
```

设置图像尺寸

```
image_shape = [480, 640]  # 高 x 宽 / Height x Width
```

定义图像的分辨率为 480x640（高度 x 宽度），后续会根据此尺寸初始化摄像头和显示模块。

初始化摄像头（RGB888 格式）

```
sensor = Sensor(id=2, width=1280, height=960, fps=30)
sensor.reset()
sensor.set_framesize(width=image_shape[1], height=image_shape[0])
sensor.set_pixformat(Sensor.RGB888)  # 设置像素格式为 RGB888 / Set pixel format to
RGB888
```

- 初始化摄像头，设置分辨率为 1280x960，帧率为 30 FPS。
- 将输出帧大小调整为 640x480，并设置为 RGB888 格式（三通道彩色图像，适合处理彩色图像的高斯滤波）。

初始化显示模块

```
Display.init(Display.ST7701, width=image_shape[1], height=image_shape[0],
to_ide=True, quality=100)
```

初始化显示模块，使用 ST7701 驱动芯片，分辨率与图像尺寸一致。`to_ide=True` 表示图像会同时传输到 IDE 进行虚拟显示，`quality=100` 设置图像传输质量为最高。

初始化媒体管理器并启动摄像头

```
MediaManager.init()
sensor.run()
```

初始化媒体资源管理器，并启动摄像头开始捕获图像流。

设置高斯滤波参数

```
kernel_size = 21
clock = time.clock()    # 启动帧率计时器 / Start FPS timer
```

- `kernel_size`：高斯滤波的卷积核尺寸，必须为奇数（如 3、5、7等），值越大模糊效果越强，但计算量增加。本代码中设置为 21，属于较大的核，会产生明显的平滑效果。
- `clock`：用于计算帧率。

图像处理与高斯滤波

```
while True:
    clock.tick()  # 开始计时 / Start timing

    # 拍摄一帧图像 / Capture a frame
    img = sensor.snapshot()
    img_np = img.to_numpy_ref()  # 获取 RGB888 图像引用 / Get RGB888 ndarray (HWC)

    # 应用高斯模糊滤波 / Apply gaussian blur using cv_lite
    denoised_np = cv_lite.rgb888_gaussian_blur_fast(
        image_shape,
        img_np,
        kernel_size
    )

    # 构造图像用于显示 / Wrap processed image for display
    img_out = image.Image(image_shape[1], image_shape[0], image.RGB888,
                          alloc=image.ALLOC_REF, data=denoised_np)

    # 显示图像 / Display the blurred image
    Display.show_image(img_out)

    # 清理内存并打印帧率 / Collect garbage and show FPS
    gc.collect()
    print("gaussian blur fps:", clock.fps())
```

- **图像捕获**：通过 `sensor.snapshot()` 获取一帧图像，并通过 `to_numpy_ref()` 将其转换为 NumPy 数组引用。

- **高斯滤波处理**：调用 `cv_lite.rgb888_gaussian_blur_fast()` 函数执行高斯滤波操作，返回处理后的图像 NumPy 数组。
- **图像包装和显示**：将处理结果包装为 RGB888 格式图像对象，并显示到屏幕或 IDE 虚拟窗口。
- **内存管理和帧率输出**：调用 `gc.collect()` 清理内存，并通过 `clock.fps()` 打印当前帧率。

资源释放

```
sensor.stop()
Display.deinit()
os.exitpoint(os.EXITPOINT_ENABLE_SLEEP)
time.sleep_ms(100)
MediaManager.deinit()
```

参数调整说明

- `kernel_size`：高斯滤波的卷积核尺寸，值越大模糊效果越强，适合处理更高水平的噪声或需要更平滑的图像；必须使用奇数（如 3、5、7等），建议从 3 或 5 开始测试，逐步增加到 21 这样的较大值以观察效果和性能影响。较大的核（如本代码中的 21）会显著增加计算量，可能降低帧率，需根据硬件性能调整。